

FastUMI Pro安装使用

官方教程：[📖 FastUMI Pro使用教程](#)

环境配置

- Ubuntu 20.04
- ROS Noetic
- 所有代码可以在<https://github.com/emia7/openpi> 获取最新代码

安装硬件sdk

代码块

```
1  git clone https://github.com/FastUMIData/FastUMI_Hardware_SDK.git
2
3  cd FastUMI_Hardware_SDK
4  cd xv/scripts/
5
6  sudo -E bash install-ros1.sh ../sdk/1212/XVSDK_focal_amd64_1212.deb
7
8  # 这里之后会自己写出一个catkin_ws
9  # sudo -E bash install-ros1.sh
   /home/arm/FastUMI_Hardware_SDK/xv/sdk/20260312/XVSDK_focal_amd64.deb
10 cd ~/catkin_ws/
11 roslaunch xv_sdk xv_sdk.launch
12
13
14
15 # 拓展USB带宽
16 #执行完后会自动重启终端
17 sudo -E bash multi-support.sh
```

更新sdk：支持头部D435i相机；减小usb传输带宽；支持压缩图像话题



- 在xv_sdk/launch中添加了D435i相机的launch文件rs_camera.launch
- 修改xv_sdk/src/xv_sdk_wrapper.cpp，支持关闭任意图像话题，支持ROS发布compressed图像话题；
- 添加录制脚本xv_sdk/scripts/record.sh

- 若使用D435i相机，则需要安装ros驱动

代码块

```
1 sudo apt-get install ros-noetic-realsense2-camera-*
```

- 从gitee下载更新的xv_sdk代码[gitee仓库]

代码块

```
1 cd ~/catkin_ws/src
2 git clone https://gitee.com/nics-robot/xv_sdk
```

- 重新编译

代码块

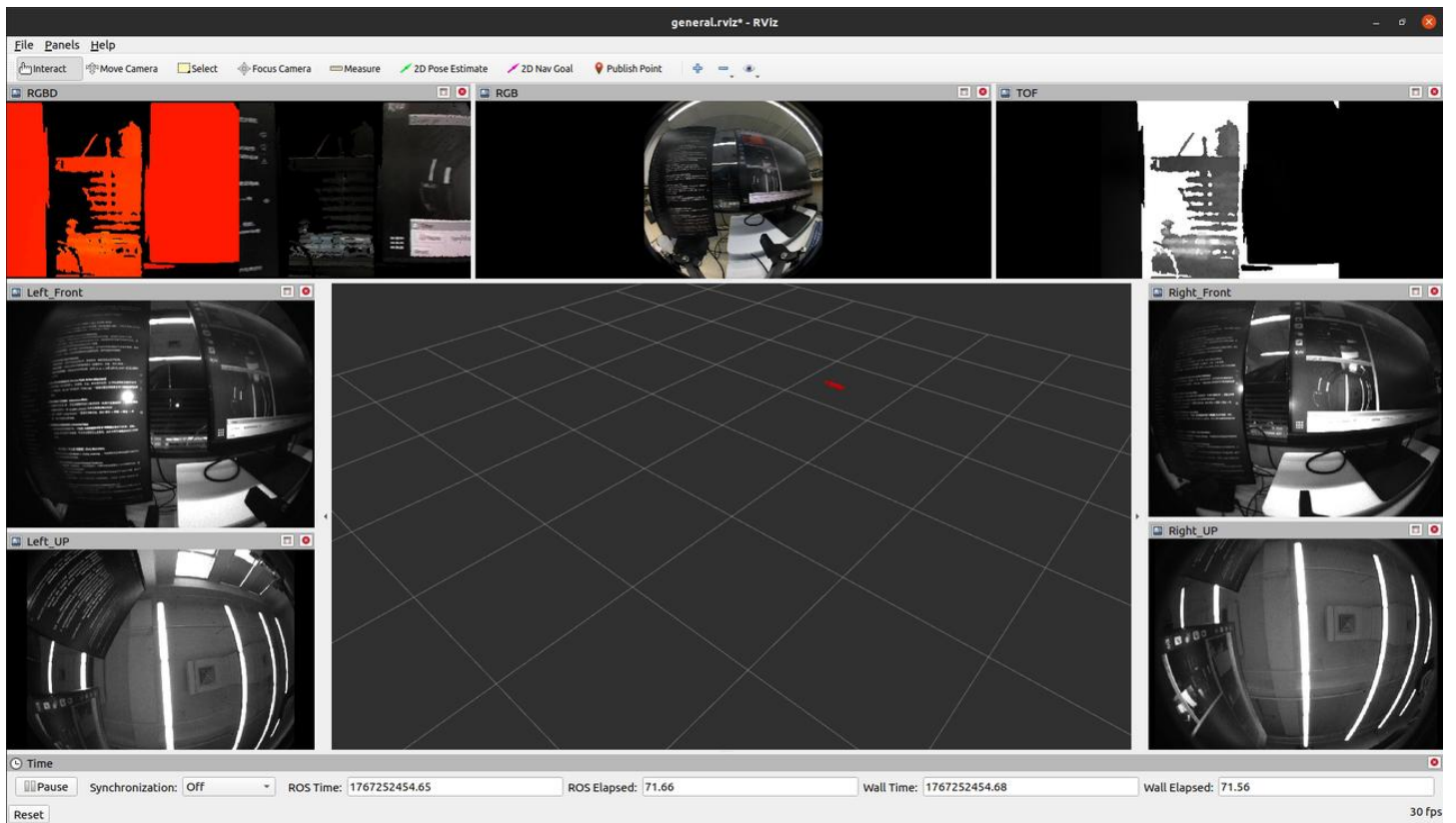
```
1 cd ~/catkin_ws
2 catkin_make -DXVSDK_INCLUDE_DIRS="/usr/include/xvsdk" -
  DXVSDK_LIBRARIES="/usr/lib/libxv sdk.so"
```

安装monitor

代码块

```
1 conda create -n fastumi python=3.8.5
2 conda activate fastumi
3
4 git clone https://github.com/FastUMIData/FastUMI_Monitor_Tool.git
5
6 pip install -r requirements.txt
7
8 roslaunch xv_sdk xv_sdk.launch
9
10 chmod +x fastumi_monitor_menu.sh
11 bash fastumi_monitor_menu.sh
```

会出来一个菜单，可以看各种传感器参数和可视化



现在相机频率只有7Hz，怀疑是usb接口问题

```
(base) nics@nics-X299-UD4-EX:~/CSW$ lsusb -t
/: Bus 06.Port 1: Dev 1, Class=root_hub, Driver=xhci_hcd/4p, 10000M
/: Bus 05.Port 1: Dev 1, Class=root_hub, Driver=xhci_hcd/2p, 480M
/: Bus 04.Port 1: Dev 1, Class=root_hub, Driver=xhci_hcd/2p, 10000M
/: Bus 03.Port 1: Dev 1, Class=root_hub, Driver=xhci_hcd/2p, 480M
/: Bus 02.Port 1: Dev 1, Class=root_hub, Driver=xhci_hcd/10p, 5000M
/: Bus 01.Port 1: Dev 1, Class=root_hub, Driver=xhci_hcd/16p, 480M
|__ Port 9: Dev 2, If 0, Class=Vendor Specific Class, Driver=rt2800usb, 480M
|__ Port 10: Dev 3, If 0, Class=Wireless, Driver=btusb, 12M
|__ Port 10: Dev 3, If 1, Class=Wireless, Driver=btusb, 12M
|__ Port 11: Dev 18, If 2, Class=Video, Driver=, 480M
|__ Port 11: Dev 18, If 0, Class=Vendor Specific Class, Driver=, 480M
|__ Port 11: Dev 18, If 3, Class=Human Interface Device, Driver=, 480M
|__ Port 11: Dev 18, If 1, Class=Video, Driver=, 480M
(base) nics@nics-X299-UD4-EX:~/CSW$
```

换用工控机解决

主相机（鱼眼） 60Hz

四个鱼眼副相机 30Hz

TOF相机 30Hz

IMU SLAM 500Hz

clamp夹爪 200Hz



当前序列号

夹爪1: 250801DR48FP25002565

相机: 250801DR48FB25002040

夹爪2: 250801DR48FP25002268

代码结构

```
===== 选择要运行的命令 (支持单个编号、逗号列表、范围) =====
1) IMU 频率      hz      slam/pose
2) IMU数据读取   echo slam/pose
3) SLAM 频率     hz      slam/pose
4) SLAM数据读取   echo slam/visual_pose
5) RGB相机频率   hz      color_camera/image
6) 左前鱼眼相机频率 hz    fisheye_cameras/left/camera_info
7) 左上鱼眼相机频率 hz    fisheye_cameras/left2/camera_info
8) 右前鱼眼相机频率 hz    fisheye_cameras/right/camera_info
9) 右上鱼眼相机频率 hz    fisheye_cameras/right2/camera_info
10) TOF相机频率   hz      tof_camera/image
11) RViz: 四鱼眼视图      four_fisheyes.rviz
12) RViz: 左前鱼眼        fisheye_left.rviz
13) RViz: 左上鱼眼        fisheye_left2.rviz
14) RViz: 右前鱼眼        fisheye_right.rviz
15) RViz: 右上鱼眼        fisheye_right2.rviz
16) RViz: RGBD 相机       rgbd_camera.rviz
17) RViz: RGB 相机       rgb_camera.rviz
18) RViz: TOF            tof.rviz
19) RViz: SLAM Pose标记   slam_pose_markers.rviz
20) RViz: 整体可视化      general.rviz
21) CLAMP 数据读取   echo clamp/Data
    0) 退出脚本2
=====
提示: 新开的终端窗口中按 Ctrl-C 可停止, 或直接关闭窗口。
EOF
}
```

```
CMD MAP[1]="rostopic hz $(TOPIC_PREFIX)/slam/pose"
CMD MAP[2]="rostopic echo $(TOPIC_PREFIX)/slam/pose"
CMD MAP[3]="rostopic hz $(TOPIC_PREFIX)/slam/pose"
CMD MAP[4]="rostopic echo $(TOPIC_PREFIX)/slam/visual_pose"
CMD MAP[5]="rostopic hz $(TOPIC_PREFIX)/color_camera/image"
CMD MAP[6]="rostopic hz $(TOPIC_PREFIX)/fisheye_cameras/left/camera_info"
CMD MAP[7]="rostopic hz $(TOPIC_PREFIX)/fisheye_cameras/left2/camera_info"
CMD MAP[8]="rostopic hz $(TOPIC_PREFIX)/fisheye_cameras/right/camera_info"
CMD MAP[9]="rostopic hz $(TOPIC_PREFIX)/fisheye_cameras/right2/camera_info"
CMD MAP[10]="rostopic hz $(TOPIC_PREFIX)/tof_camera/image"
CMD MAP[11]="rviz -d $(RVIZ_PATH)/four_fisheyes.rviz"
CMD MAP[12]="rviz -d $(RVIZ_PATH)/fisheye_left.rviz"
CMD MAP[13]="rviz -d $(RVIZ_PATH)/fisheye_left2.rviz"
CMD MAP[14]="rviz -d $(RVIZ_PATH)/fisheye_right.rviz"
CMD MAP[15]="rviz -d $(RVIZ_PATH)/fisheye_right2.rviz"
CMD MAP[16]="rviz -d $(RVIZ_PATH)/rgbd_camera.rviz"
CMD MAP[17]="rviz -d $(RVIZ_PATH)/rgb_camera.rviz"
CMD MAP[18]="rviz -d $(RVIZ_PATH)/tof.rviz"
CMD MAP[19]="rviz -d $(RVIZ_PATH)/slam_pose_markers.rviz"
CMD MAP[20]="rviz -d $(RVIZ_PATH)/general.rviz"
CMD MAP[21]="rosservice call $(TOPIC_PREFIX)/clamp/stop;rosservice call $(TOPIC_PREFIX)/clamp/start;rostopic echo $(TOPIC_PREFIX)/clamp/Data"
```

```
(base) abc@Ubuntu:~/fastumi/FastUMI_Monitor_Tool$ rostopic list
/rosout
/rosout_agg
/tf
/tf_static
/xv_sdk/250801DR48FP25002565/clamp/Data
/xv_sdk/250801DR48FP25002565/color_camera/camera_info
/xv_sdk/250801DR48FP25002565/color_camera/image
/xv_sdk/250801DR48FP25002565/color_camera/point_cloud
/xv_sdk/250801DR48FP25002565/fisheye_cameras/left/camera_info
/xv_sdk/250801DR48FP25002565/fisheye_cameras/left/image
/xv_sdk/250801DR48FP25002565/fisheye_cameras/left2/camera_info
/xv_sdk/250801DR48FP25002565/fisheye_cameras/left2/image
/xv_sdk/250801DR48FP25002565/fisheye_cameras/right/camera_info
/xv_sdk/250801DR48FP25002565/fisheye_cameras/right/image
/xv_sdk/250801DR48FP25002565/fisheye_cameras/right2/camera_info
/xv_sdk/250801DR48FP25002565/fisheye_cameras/right2/image
/xv_sdk/250801DR48FP25002565/imu_sensor/data_raw
/xv_sdk/250801DR48FP25002565/parameter_descriptions
/xv_sdk/250801DR48FP25002565/parameter_updates
/xv_sdk/250801DR48FP25002565/rgbd_camera/camera_info
/xv_sdk/250801DR48FP25002565/rgbd_camera/image
/xv_sdk/250801DR48FP25002565/slam/pose
/xv_sdk/250801DR48FP25002565/slam/pose_markers
/xv_sdk/250801DR48FP25002565/tof_camera/camera_info
/xv_sdk/250801DR48FP25002565/tof_camera/image
/xv_sdk/new_device
/xv_sdk/parameter_descriptions
/xv_sdk/parameter_updates
```

需要数据:

/tf

/tf_static

/xv_sdk/250801DR48FP25002565/clamp/Data

/xv_sdk/250801DR48FP25002565/color_camera/camera_info

/xv_sdk/250801DR48FP25002565/color_camera/image

/xv_sdk/250801DR48FP25002565/color_camera/point_cloud

/xv_sdk/250801DR48FP25002565/rgbd_camera/camera_info

/xv_sdk/250801DR48FP25002565/rgbd_camera/image

/xv_sdk/250801DR48FP25002565/slam/pose

/xv_sdk/250801DR48FP25002565/slam/pose_markers

单臂任务

采数据

Terminal 1

代码块

```
1
2 cd ~/catkin_ws/
3 roslaunch xv_sdk xv_sdk.launch
```

Terminal 2

代码块

```
1  conda activate fastumi
2
3  cd ./FastUMI_Monitor_Tool/
4
5  bash fastumi_monitor_menu.sh
6
7  20
```

先在terminal 2 中打开21，再采

Terminal 3

代码块

```
1 cd ~/CSW/data
2
3 rosbag record \
4   /xv_sdk/250801DR48FP25002565/clamp/Data \
5   /xv_sdk/250801DR48FP25002565/color_camera/image \
6   /xv_sdk/250801DR48FP25002565/slam/pose
```

```
cd ~/CSW/data
```

```
rosbag record /xv_sdk/250801DR48FP25002268/clamp/Data
/xv_sdk/250801DR48FP25002268/color_camera/image
/xv_sdk/250801DR48FP25002268/slam/pose
```

采完写一个readme，记下序列号，任务，数目，采集时间

无本体PI0.5 finetune

<https://github.com/Physical-Intelligence/openpi?tab=readme-ov-file#installation>

代码块

```
1  # 拉ckpt
2
3  from openpi.training import config as _config
4  from openpi.policies import policy_config
5  from openpi.shared import download
6
7  config = _config.get_config("pi05_droid")
8  checkpoint_dir = download.maybe_download("gs://openpi-
  assets/checkpoints/pi05_base")
```

数据格式转换

1. rosbag 2 mp4+json

在本机不开conda环境中进行(需要import rosbag)

代码块

```
1  python convert_ros_data_to_mp4.py --bag /home/nics/Downloads/2026-01-02-22-27-
  59.bag --serial 250801DR48FP25002565 --out_dir
  /home/nics/CSW/openpi/umi/data --data_idx=1
```

处理后文件夹下一个mp4，一个json

批量处理

代码块

```
1  ./batch_stage1_vis.sh
```

数据处理：检查一下mp4和json有没有不对劲的，比如pose没录到或者全0，或者是某些数据明显不对
删除不对劲的之后做重命名

代码块 先预览一下

```
2 python .\openpi\umi\json_sort.py --dir
  .\batch_pickup_cubes_0125\batch_pickup_cubes_0125\
3
4 # 没问题就 --apply
5 python .\openpi\umi\json_sort.py --dir
  .\batch_pickup_cubes_0125\batch_pickup_cubes_0125\ --apply
```

随后将mp4和json scp过去

代码块

```
1 scp /home/nics/CSW/openpi/umi/data.zip eva14:/share/chenshuaiwen-
  local/umi_data_raw
```

2. mp4+json 2 lerobot

服务器实现，需要uv的lerobot信息

state ["x", "y", "z", "qx", "qy", "qz", "qw", "clamp"]

现在给出的是绝对位置

chenshuaiwen-local下umi_data_raw

代码块

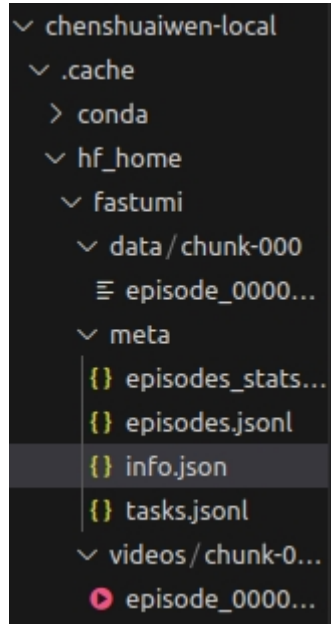
```
1 unzip data.zip
```

现在将数据降采样到10Hz

代码块

```
1 python convert_mp4_data_to_lerobot_downsample.py --
  stage1_dir=/share/chenshuaiwen-local/umi_data_raw/batch_200 --
  repo=/share/chenshuaiwen-local/.cache/hf_home/fastumi --target_fps=10
```

结果格式



info.json

代码块

```
1  {
2      "codebase_version": "v2.1",
3      "robot_type": "XV",
4      "total_episodes": 1,
5      "total_frames": 317,
6      "total_tasks": 1,
7      "total_videos": 1,
8      "total_chunks": 1,
9      "chunks_size": 1000,
10     "fps": 60,
11     "splits": {
12         "train": "0:1"
13     },
14     "data_path": "data/chunk-
{episode_chunk:03d}/episode_{episode_index:06d}.parquet",
15     "video_path": "videos/chunk-
{episode_chunk:03d}/{video_key}/episode_{episode_index:06d}.mp4",
16     "features": {
17         "wrist_view": {
18             "dtype": "video",
19             "shape": [
20                 1280,
21                 1280,
22                 3
23             ],
24             "names": [
25                 "height",
26                 "width",
```

```
27         "channel"
28     ],
29     "info": {
30         "video.height": 1280,
31         "video.width": 1280,
32         "video.codec": "av1",
33         "video.pix_fmt": "yuv420p",
34         "video.is_depth_map": false,
35         "video.fps": 60,
36         "video.channels": 3,
37         "has_audio": false
38     }
39 },
40 "state": {
41     "dtype": "float32",
42     "shape": [
43         8
44     ],
45     "names": [
46         "x",
47         "y",
48         "z",
49         "qx",
50         "qy",
51         "qz",
52         "qw",
53         "clamp"
54     ]
55 },
56 "actions": {
57     "dtype": "float32",
58     "shape": [
59         8
60     ],
61     "names": [
62         "x",
63         "y",
64         "z",
65         "qx",
66         "qy",
67         "qz",
68         "qw",
69         "clamp"
70     ]
71 },
72 "timestamp": {
73     "dtype": "float32",
```

```

74         "shape": [
75             1
76         ],
77         "names": null
78     },
79     "frame_index": {
80         "dtype": "int64",
81         "shape": [
82             1
83         ],
84         "names": null
85     },
86     "episode_index": {
87         "dtype": "int64",
88         "shape": [
89             1
90         ],
91         "names": null
92     },
93     "index": {
94         "dtype": "int64",
95         "shape": [
96             1
97         ],
98         "names": null
99     },
100    "task_index": {
101        "dtype": "int64",
102        "shape": [
103            1
104        ],
105        "names": null
106    }
107 }
108 }

```

replay

两个发现：

1. ~~fastumi初始的坐标系和franka不一样，坐标轴朝向不同，需要在启动时保持两者坐标朝向相同~~
2. 夹爪给 -1 才能夹紧

代码块

```

1  # 真机正常跑
2  ./replay_data_fastumi_relative.sh pick-place-0 1

```

训练

1. 对齐输入格式

src/openpi/policies下 xv_policy

代码块

```
1  现在有了新的umi_hybrid_policy
```

2. 改写config文件

需要一个data process config， 一个train config

train config写了数据路径和base model路径，还有一些训练参数

注意ckpt不要保存在home下，有内存限额，在config的

checkpoint_base_dir: str = "share/chenshuaiwen-local/checkpoints" 修改

3. 计算norm

原版的norm是一个chunk内的所有action使用相同方式做norm（如有问题，确保创建数据数据集和计算norm时，numpy版本为1.26.4 datasets版本为3.6.0）

代码块

```
1  uv run scripts/compute_norm_stats.py --config-name pi05_xv_finetune
```

代码块

```
1  {
2      "norm_stats": {
3          "state": {
4              "mean": [
5                  0.1050405502319336,
6                  -0.09088267385959625,
7                  -0.524502694606781,
8                  -0.0047263880260288715,
9                  0.5114215016365051,
10                 0.009156366810202599,
11                 0.8567602634429932,
```

```
12      73.65776824951172
13    ],
14    "std": [
15      0.07411293685436249,
16      0.025313572958111763,
17      0.06006355583667755,
18      0.026961518451571465,
19      0.04982772096991539,
20      0.015806788578629494,
21      0.029054785147309303,
22      16.016319274902344
23    ],
24    "q01": [
25      -0.04216261932551861,
26      -0.16013982579410077,
27      -0.6028686562776565,
28      -0.05367918088883161,
29      0.41491750063896177,
30      -0.02851395530253649,
31      0.8104992325782776,
32      51.95542596282959
33    ],
34    "q99": [
35      0.23673565061986446,
36      -0.06401133262217044,
37      -0.3702419506311416,
38      0.054312397417426106,
39      0.5843053293704986,
40      0.0435803895637393,
41      0.9090512665748596,
42      87.83663453903199
43    ]
44  },
45  "actions": {
46    "mean": [
47      0.10613217204809189,
48      -0.09064702689647675,
49      -0.5266942381858826,
50      -0.004248251672834158,
51      0.5140153169631958,
52      0.009151234291493893,
53      0.8552988171577454,
54      73.65213775634766
55    ],
56    "std": [
57      0.07306119054555893,
58      0.025427717715501785,
```

```

59         0.05987371504306793,
60         0.02698267623782158,
61         0.04871905595064163,
62         0.015458771958947182,
63         0.02845783345401287,
64         16.011730194091797
65     ],
66     "q01": [
67         -0.040666409796476365,
68         -0.16013982579410077,
69         -0.602821064889431,
70         -0.05367918088883161,
71         0.4213169141769409,
72         -0.02851395530253649,
73         0.8104992325782776,
74         51.96260938796997
75     ],
76     "q99": [
77         0.23673837686404586,
78         -0.06489471894204615,
79         -0.3702419506311416,
80         0.0543359866656363,
81         0.5843053293704986,
82         0.04324602974876762,
83         0.9056673264503479,
84         87.82226768875123
85     ]
86 }
87 }
88 }

```

TODO: 为了满足relative action，需要考虑在chunk的每一个T分别做norm

4. 开训

注意开tmux，现在的tmux和exp_name相同

代码块

```

1  tmux new -s exp_name
2
3  tmux attach -t exp_name

```

注意改exp_name

JAX会默认使用可见的所有卡，所以需要修改可见卡数为当前可用的

代码块

```
1  CUDA_VISIBLE_DEVICES=0 XLA_PYTHON_CLIENT_MEM_FRACTION=0.9 uv run
   scripts/train.py pi05_xv_finetune --exp-name=0107 --overwrite
```

注意一下loss情况

训完把norm_stats.json放在assets下

5. offline测试，选验证集进行离线测试，看与GT的差距

注意修改exp-name, step, repo, episode

代码块

```
1  CUDA_VISIBLE_DEVICES=4 XLA_PYTHON_CLIENT_MEM_FRACTION=0.9 uv run python
   ./umi/eval_relative_new.py --config pi05_xv_finetune --exp-name 0119 --
   step 29999 --repo /share/chenshuaiwen-
   local/.cache/hf_home/fastumi/0119_batch_data/ --episode 0 --out-dir
   eval_results/relative/0 --dims 10 --use-time-axis
```

部署

- openpi里起server，openpi的配置与前面相同，需要写xv_policy和config，config里写到assets
 - serve_policy 里面可以指定端口号，默认prompt等

代码块

```
1  # openpi下,注意这里指定config和路径
2  # 注意ckpt的assets下应有一个norm_stats.json
3  uv run scripts/serve_policy.py policy:checkpoint --
   policy.config=pi05_xv_finetune --
   policy.dir=/home/m/share/pi05ckpt/ckpt_0104_test/2000
```

- 目前修改了wesocket_policy_server，对齐eval输出和模型输入接口
- 本地跑eval，使用realRL `scripts/eval_policy.sh`，
在 `/home/abc/qiuyi/realRL/experiments/sq_franka_base/tasks/fastumipro_`
`dishwashing` 下面建了任务
 - realRL 可以指定任务名，policy host IP，端口号，配置任务

添加任务方法：

To add new tasks for an existing robot (e.g., `souhu_droid_base`):

1. Create a task group directory:

```
experiments/souhu_droid_base/tasks/my_new_tasks/
```

2. Add `tasks.json` defining the tasks:

```
{ "task_key_1": "Description of task 1", "task_key_2": "Description of task 2"
}
```

4. 在base文件夹下添加config文件

(Optional) Add `subtasks.json` for fine-grained progress tracking.

代码块

```
1 ./run_server.sh
```

代码块

```
1 # realRL
2 ./scripts/eval_policy.sh 20 pick-place-0
```

数据分析

粗略比较：

代码块

```
1 python .\compare_batches.py --dir_a
  C:\Users\LENOVO\Downloads\umi_pick_place_cup_0122_200episodes\batch_200 --
  dir_b
  C:\Users\LENOVO\Downloads\umi_pick_place_cup_0209_100episodes\pick_place_cup_ne
  w
```

会给一个数据整体信息的json文件和一些数据分布图

类似于：

A是csw的200条，B是龙哥的100条

长度均值：A 258，B 299

timestamp fps: A 60.26, B 60.47

Δ pos p99 median: A 0.0131m, B 0.0092m

→ A 的平移步长约大 1.43 倍

Δ rot p99 median: A 0.0269rad, B 0.0121rad

→ A 的旋转步长约大 2.2 倍

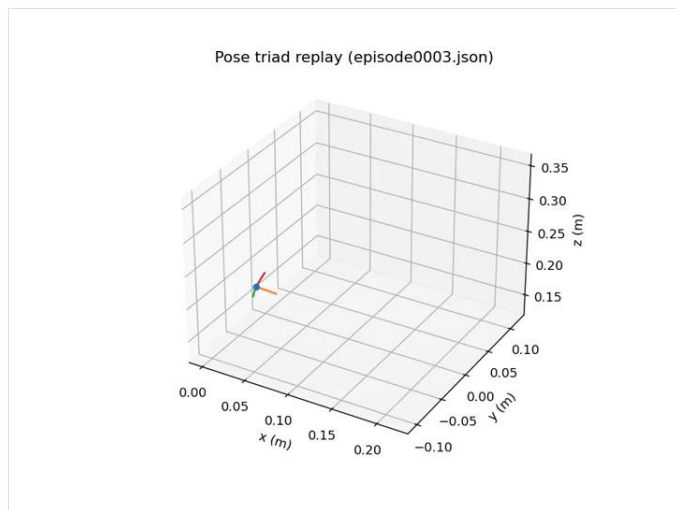
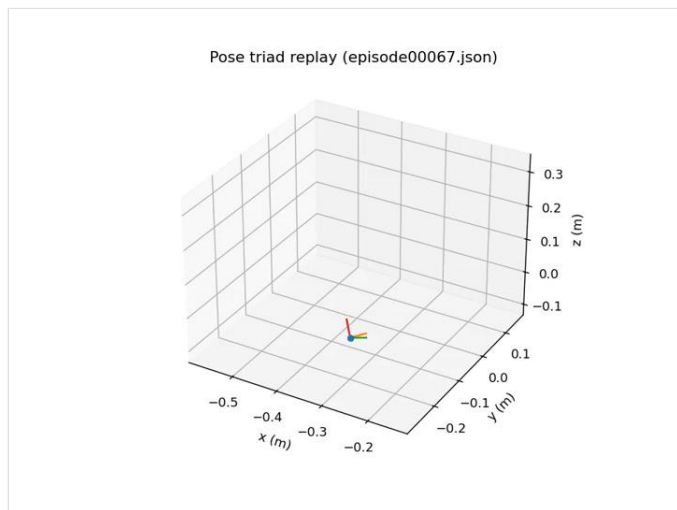
可视化pose

代码块

```
1 python .\render_triad_mp4.py --json
  C:\Users\LENOVO\Downloads\umi_pick_place_cup_0122_200episodes\batch_200\episode
  00067.json --out csw_3.mp4
```

得到一个三维空间中的pose移动图像

类似于



带transform后的图像可视化和所有chunk记录：

代码块

```
1 CUDA_VISIBLE_DEVICES=4 XLA_PYTHON_CLIENT_MEM_FRACTION=0.9 uv run python
  ./umi_scripts_csw/eval_relative_visualize.py --config
  pi05_umi_rel6d_image_only_finetune --exp-name 0224_s3 --step 19999 --repo
  /share/chenshuaiwen-local/.cache/hf_home/fastumi/pick_place_csw_umi_hybrid/ -
  -episode 1 --out-dir eval_results/relative/0224_s3/1
```

```

代码块
CUDA_VISIBLE_DEVICES=4 XLA_PYTHON_CLIENT_MEM_FRACTION=0.9 uv run python
./umi_scripts_csw/eval_relative_visualize.py --config pi05_xv_finetune --
exp-name 0224_s1 --step 20000 --repo /share/chenshuaiwen-
local/.cache/hf_home/fastumi/pick_place_csw/ --episode 1 --out-dir
eval_results/relative/0224_s1/1

```

算是一种offline，会将训练用的chunk内容会存到npz里面

图像有一个mp4，是降采样后的



比较存下的几个npz文件

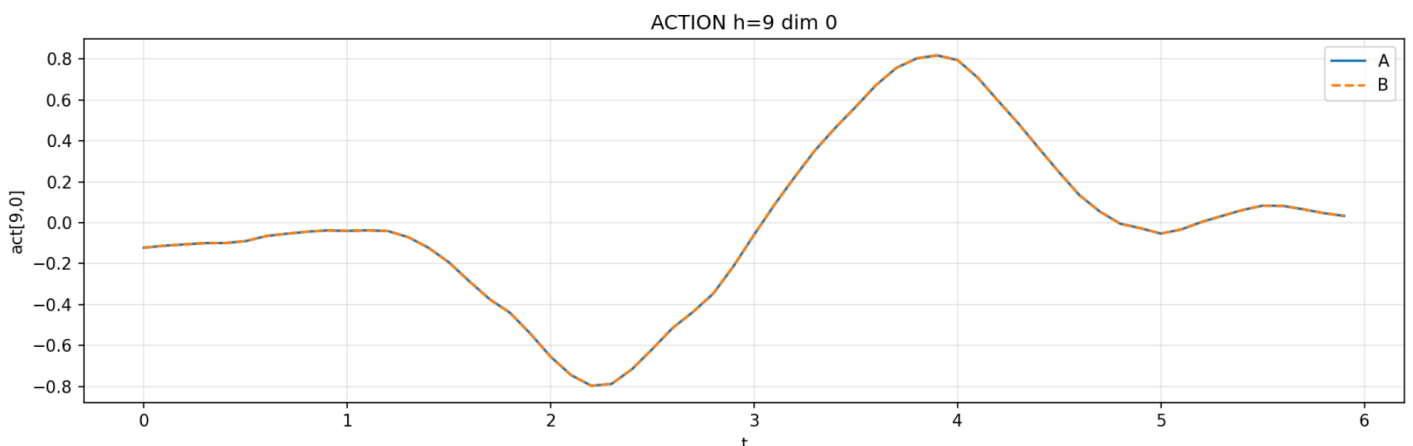
代码块

```

1 uv run python ./umi_scripts_csw/compare_npz.py --a
/home/chenshuaiwen/openpi/eval_results/relative/0224_s1/1/ep0001_transformed_in
puts.npz --b
/home/chenshuaiwen/openpi/eval_results/relative/0224_s3/1/ep0001_transformed_in
puts.npz --out ./compare_npz_out/0224s1_0224s3/1 --save-action-heatmaps

```

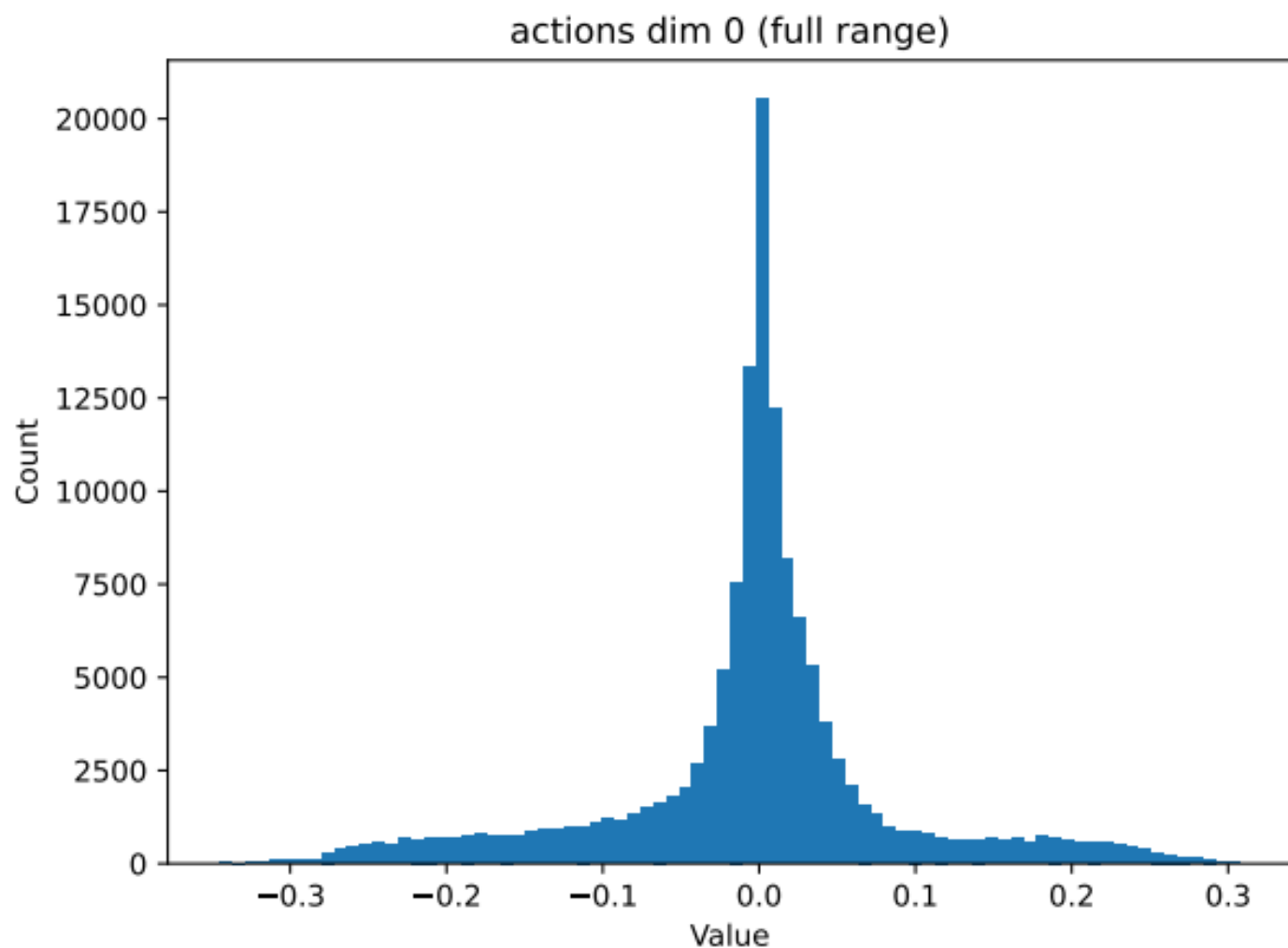
会以图像形式给出所有的action情况，共(horizon,dim)张，类似于下图



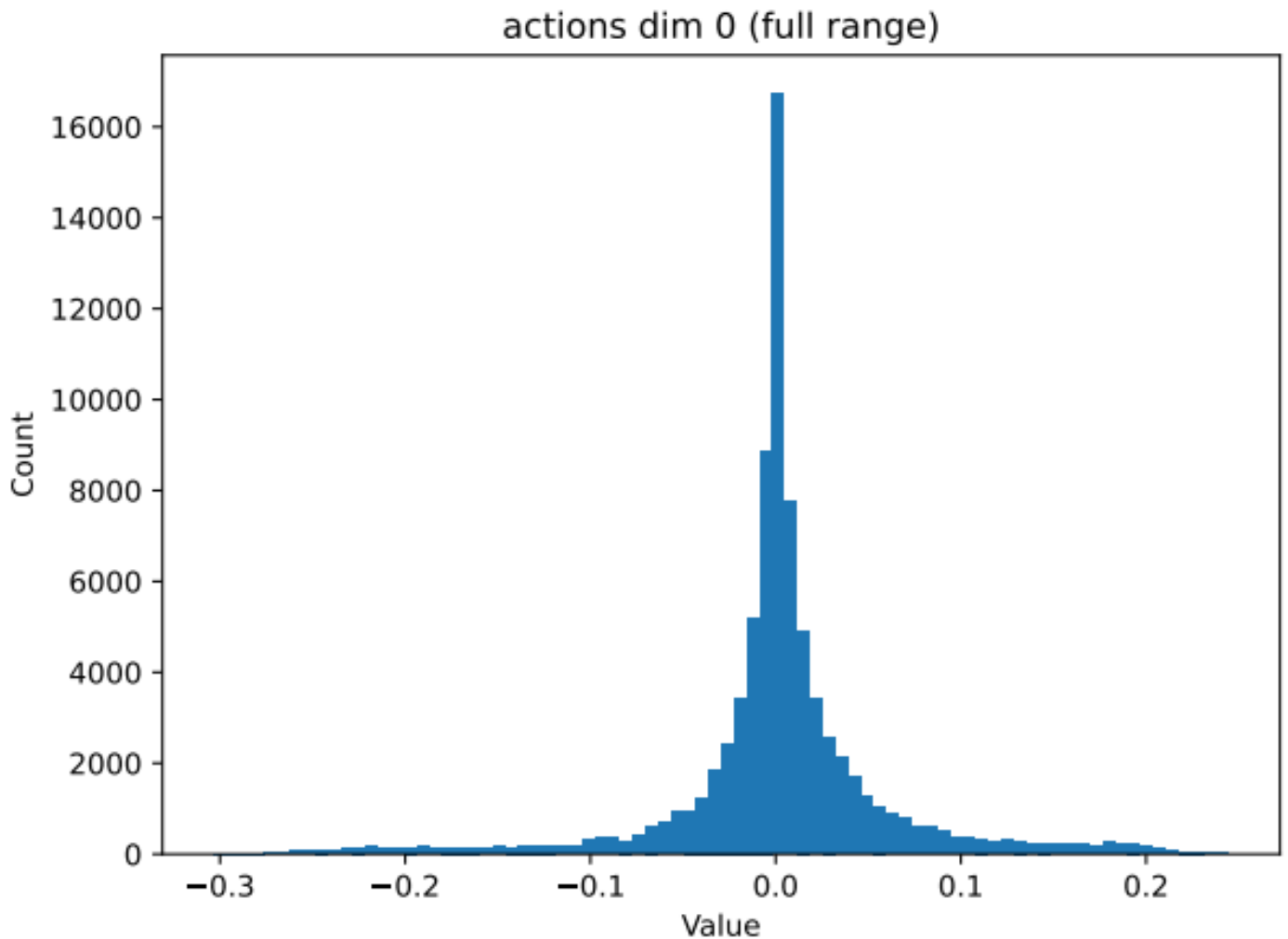
可视化action的分布情况

代码块

```
1 uv run scripts/compute_norm_stats_w_image.py --config-name pi05_xv_finetune
```



 actions_fullrange_histograms_csw.pdf



 actions_fullrange_histograms_long.pdf

take-away

1. relative在于somehow消除掉base坐标系
2. 图像的输入频率很重要 ???
3. 每批数据都需要抽查做replay，或者隔一段时间抽查看state分布情况
4. loss其实没啥用，只能看到输入输出有没有对齐，还得看offline的测试结果
5. retry和干扰数据很重要

双臂

暂定使用鹿鸣的数据采集平台来做双臂

使用Chrome浏览器，有什么问题就插拔硬件和重启设备（检查是否3.0USB接口）

鹿明环境

代码块

```
1 roslaunch xv_sdk xv_sdk.launch
2
3 sudo -E bash multi-support.sh
```

采数据：

1. 打开数据采集平台后端：

代码块

```
1 cd /home/ubuntu/qiuyi/FastUMI/FastUMI_Data_Platform_Web
2 bash ./deploy.sh
3
```

2. chrome开数据采集平台前端

<http://localhost:8000/data-collector>

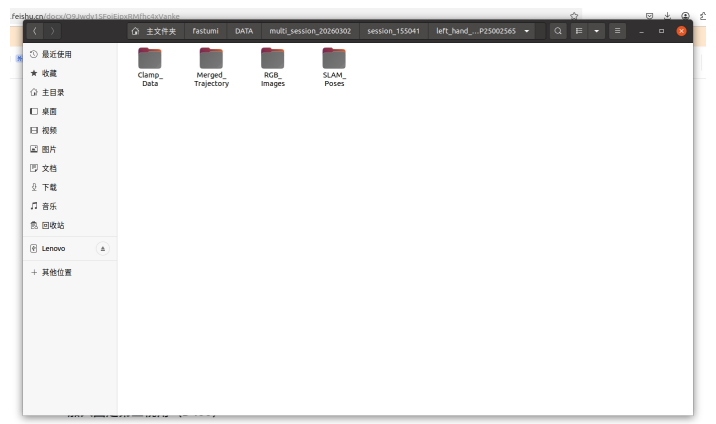
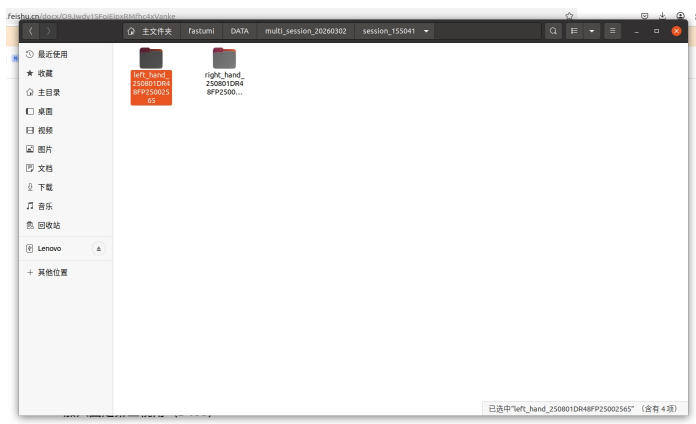
3. 将两个设备放到一个相对固定的位置，开启设备，连接设备，开始采集

如果报错，关闭设备重新开启或者插拔一下USB

4. 看一下图像频率能不能跑到60Hz，打出rostopic看一下

5. 开始采集

采集后数据格式为



视频为60Hz mp4，其他为txt

为了和上面内容对齐，我们转化为mp4+json

采数据2:

采集平台两个设备是分开采集和训练的，其实用不用无所谓，感觉也不如直接用roslaunch录制运行

代码块

```
1
2 cd ~/catkin_ws/
3
4 roslaunch xv_sdk xv_sdk.launch
5 # 启动fastumi
```

两个分别打开monitor来看

代码块

```
1 cd ./FastUMI_Monitor_Tool/
2
3 bash fastumi_monitor_menu.sh
4
5 21
6
7 20
```

开第三视角的D435

代码块

```
1 # 开D435
2
3 roslaunch realsense2_camera rs_camera.launch
4
5 # 可以看 rqt_image_view 来看现在的视频
```

看topic信息

代码块

```
1 rostopic list
2
3 # 打印image topic的频率
4 rostopic hz xxx
```

- 5 # 注意到两个相机都能跑到60Hz，录制时右手相机会掉帧，猜测是录制的带宽问题，尝试能不能直接录制10Hz内容

开一个脚本，订阅现在的几个图像信息，降采样到10Hz之后进行录制

代码块

```
1 # 把脚本放进某个 catkin 包的 scripts/ (比如 FastUMI 里你已有的包)
2 chmod +x scripts/tri_image_sampler_10hz.py
3 rosrun <your_pkg> tri_image_sampler_10hz.py
4
5 # python脚本，不需要重新编译，但需要放到一个package里面，保证能rosrun出来
6 # 理论上这个脚本会得到_10hz的一系列topic
```

直接用rosbag录制

代码块

```
1 mkdir -p ~/qiuyi/FastUMI/data/bags
2 cd ~/qiuyi/FastUMI/data
3
4 rosbag record -O bags/all_10hz_$(date +%Y%m%d_%H%M%S).bag \
5   /xv_sdk/250801DR48FP25002268/color_camera/image_10hz \
6   /xv_sdk/250801DR48FP25002565/color_camera/image_10hz \
7   /camera/color/image_raw/compressed_10hz \
8   /xv_sdk/250801DR48FP25002268/slam/pose \
9   /xv_sdk/250801DR48FP25002268/clamp/Data \
10  /xv_sdk/250801DR48FP25002565/slam/pose \
11  /xv_sdk/250801DR48FP25002565/clamp/Data \
12  --lz4 --buffsize=2048 --chunksize=512
13 # 平动
14 rosbag record -O bags/all_10hz_$(date +%Y%m%d_%H%M%S).bag \
15   /xv_sdk/250801DR48FP25002650/color_camera/image_10hz \
16   /xv_sdk/250801DR48FP26001203/color_camera/image_10hz \
17   /camera/color/image_raw/compressed_10hz \
18   /xv_sdk/250801DR48FP25002650/slam/pose \
19   /xv_sdk/250801DR48FP25001203/clamp/Data \
20   /xv_sdk/250801DR48FP2500265/slam/pose \
21   /xv_sdk/250801DR48FP25002565/clamp/Data \
22   --lz4 --buffsize=2048 --chunksize=512
23
```


采数据3



catkin_dual.zip

251.10KB



启动命令太复杂了，给一个rospackage

在conda外面运行（192.168.20.43）

代码块

```
1 ./start_bringup.sh
2
```

两个分别打开monitor来看

代码块

```
1 cd ./FastUMI_Monitor_Tool/
2
3 bash fastumi_monitor_menu.sh
4
5 21
6
7 2
8
```

建议把clamp和pose挂着看情况

直接用rosbag录制

代码块

```
1 mkdir -p ~/qiuyi/FastUMI/data/bags
2 cd ~/qiuyi/FastUMI/data
3
4 rosbag record -O bags/all_10hz_$(date +%Y%m%d_%H%M%S).bag \
5 /xv_sdk/250801DR48FP25002268/color_camera/image_10hz \
6 /xv_sdk/250801DR48FP25002565/color_camera/image_10hz \
7 /camera/color/image_raw/compressed_10hz \
8 /xv_sdk/250801DR48FP25002268/slam/pose \
9 /xv_sdk/250801DR48FP25002268/clamp/Data \
10 /xv_sdk/250801DR48FP25002565/slam/pose \
11 /xv_sdk/250801DR48FP25002565/clamp/Data \
12 --lz4 --buffsize=2048 --chunksize=512
13
```

rqt_image_view

数据格式转换

1. rosbag ——> mp4+json

代码块

```
1  # 有一个123的脚本和对应的bash文件
2
3  chmod +x batch_stage1_vis_123.sh
4  ./batch_stage1_vis_123.sh /path/to/bags /path/to/output start_idx
   convert_rosbag_to_mp4_vis_123.py
5
6  # ./umi_scripts_csw/batch_stage1_vis_123.sh
   /home/ubuntu/qiuyi/FastUMI/data/bags
   /home/ubuntu/qiuyi/FastUMI/data/bags/stage1 1
```

2. 查看是否有明显不对劲的state，有的话删去，对当前文件夹内容做一下sort

代码块

```
1  python3 json_sort_123.py --dir /path/to/output --start 0
```

随后将mp4和json scp过去

代码块

```
1  scp /home/nics/CSW/openpi/umi/data.zip eva14:/share/chenshuaiwen-
   local/umi_data_raw
```

3. mp4 + json 转成lerobot

代码块

```
1  python convert_mp4_data_to_lerobot_123.py --stage1_dir=/share/chenshuaiwen-
   local/umi_data_raw/batch_200 --repo=/share/chenshuaiwen-
   local/.cache/hf_home/fastumi
```

replay

暂时挂起

训练

1. 对齐输入格式

src/openpi/policies下 xv_dual_policy

2. 改写config文件，使用pi05_xv_dual_finetune

注意ckpt不要保存在home下，有内存限额，在config的

checkpoint_base_dir: str = "share/chenshuaiwen-local/checkpoints" 修改

3. 计算norm

原版的norm是一个chunk内的所有action使用相同方式做norm

代码块

```
1 uv run scripts/compute_norm_stats.py --config-name pi05_xv_dual_finetune
```

4. 开训

注意开tmux，现在的tmux和exp_name相同

代码块

```
1 tmux new -s exp_name
2
3 tmux attach -t exp_name
```

注意改exp_name

JAX会默认使用可见的所有卡，所以需要修改可见卡数为当前可用的

代码块

```
1 CUDA_VISIBLE_DEVICES=0 XLA_PYTHON_CLIENT_MEM_FRACTION=0.9 uv run
  scripts/train.py pi05_xv_dual_finetune --exp-name=0303 --overwrite
```

注意一下loss情况

训完把norm_stats.json放在assets下

offline测试

eval_dual_relative.py

代码块

```
1  # 对比前4个时间步
2  uv run python umi_scripts_csw/eval_dual_relative.py \
3      --config=pi05_xv_dual_finetune \
4      --exp-name=your_exp \
5      --step=20000 \
6      --repo=/path/to/data \
7      --episode=0 \
8      --compare-steps="0,1,2,3"
9
10 # 对比所有时间步（假设horizon=10）
11 uv run python umi_scripts_csw/eval_dual_relative.py \
12     ... \
13     --compare-steps="all"
```

部署

与gello策略相同

openpi部分

openpi里起server，openpi的配置与前面相同，

- serve_policy 里面可以指定端口号，默认prompt等

代码块

```
1  # openpi下,注意这里指定config和路径
2  # 注意ckpt的assets下应有一个norm_stats.json
3  uv run scripts/serve_policy_dual.py policy:checkpoint --policy.config=
    {your_config_name} --policy.dir={your_policy_dir}
```

- 目前修改了wesocket_policy_server，对齐eval输出和模型输入的接口

相机发布

左臂

发布第三视角相机图像

先获取相机标识：

代码块

```
1 # realsense
2 python scripts/get_serial_num.py
3
4 # lumos
5 ls /dev/v4l/by-id/
```

代码块

```
1 # 修改serial部分
2 python scripts/camera_zmq_pub.py \
3     --name third_d455 --type realsense --serial 941322072906 \
4     --port 7000 --width 640 --height 480 --fps 30 --hz_pub 30 \
5     --out_width 420 --out_height 240 \
6     --jpeg_quality 80
```

发布腕部相机图像

代码块

```
1 # lumos
2 python scripts/camera_zmq_pub.py \
3     --name left_wrist_lumos --type lumos --serial usb-
4     XVisio_Technology_XVisio_vSLAM_250801DR48FB26001173-video-index0 \
5     --port 7001 --width 1280 --height 1280 --fps 60 --hz_pub 30 \
6     --out_width 420 --out_height 240 \
7     --jpeg_quality 80
```

代码块

```
1 # realsense, 同样更换--serial内容
2 python scripts/camera_zmq_pub.py \
3     --name left_wrist_d435 --type realsense --serial 141722078696 \
4     --port 7002 --width 640 --height 480 --fps 30 --hz_pub 30 \
5     --out_width 420 --out_height 240 \
6     --jpeg_quality 80
```

右臂

发布腕部相机图像

代码块

```
1 python scripts/camera_zmq_pub.py \  
2     --name right_wrist_lumos --type lumos --serial usb-  
   XVisio_Technology_XVisio_vSLAM_250801DR48FB26001218-video-index0 \  
3     --port 7003 --width 1280 --height 1280 --fps 60 --hz_pub 30 \  
4     --out_width 420 --out_height 240 \  
5     --jpeg_quality 80
```

代码块

```
1 python scripts/camera_zmq_pub.py \  
2     --name right_wrist_d435 --type realsense --serial 141722078696 \  
3     --port 7004 --width 640 --height 480 --fps 30 --hz_pub 30 \  
4     --out_width 420 --out_height 240 \  
5     --jpeg_quality 80
```

eval

本地跑eval, 使用realRL `scripts/eval_policy.sh`

注意本地控制器需要使用relative的控制, 搜索"relative" / "delta" 查看位置, 进行修改

代码块

```
1 ./run_server.sh
```

代码块

```
1 # realRL, 默认使用lumos相机  
2 ./scripts/eval_policy.sh {task_name} 20  
3  
4 # --zmq_policy_image_keys=left_wrist_lumos,right_wrist_lumos,third_d455  
5 # 可以使用这个参数进行相机修改, 建议写在sh里面
```

数据分析

加入固定第三视角（D435）

以_13结尾的脚本是对单臂第三视角的适配

采数据

Terminal 1

代码块

```
1
2  cd ~/catkin_ws/
3  roslaunch xv_sdk xv_sdk.launch
```

Terminal 2

代码块

```
1  conda activate fastumi
2
3  cd ./FastUMI_Monitor_Tool/
4
5  bash fastumi_monitor_menu.sh
6
7  20
```

先在terminal 2 中打开21，再采

Terminal 3

代码块

```
1  # 开D435
2
3  roslaunch realsense2_camera rs_camera.launch
4
5  # 可以看 rqt_image_view 来看现在的视频
```

Terminal 4

代码块

```
1  cd ~/CSW/data
2
3  rosbag record  /camera/color/image_raw/compressed
    /xv_sdk/250801DR48FP25002268/clamp/Data
```

```
/xv_sdk/250801DR48FP25002268/color_camera/image
/xv_sdk/250801DR48FP25002268/slam/pose
4
5 # 采完写一个readme, 记下序列号, 任务, 数目, 采集时间
```

数据格式转换

1. rosbag 2 mp4

代码块

```
1 python convert_rosbag_to_mp4_vis_13.py --bag /path/to/your.bag --serial
  250801DR48FP25002268 --out_dir ./output --data_idx 0
2
```

批量化处理

代码块

```
1 ./batch_stage1_vis_13.sh
2 # 有一个可选参数, start_idx, 用来处理增量式新加数据, 默认为0
3 # 如果为50, 命名就从episode0050开始
```

采集时电脑内存不够:

解决方案1: 采一部分, 然后存到硬盘, 全部采完一起转mp4+json

解决方案2: 采一部分, 转成MP4+json, 之后继续采, 转mp4+json时修改start_idx即可

注意到一些数据的可视化图明显不对劲, 选择直接删除这部分数据, 最终对整个文件夹进行整理

代码块

```
1 # 预览整理结果
2 python json_sort_13.py --dir ./data_processed_vis
3 # 实际执行
4 python json_sort_13.py --dir ./data_processed_vis --apply
5
```

随后将mp4和json scp过去


```
1 scp /home/nics/CSW/openpi/umi/data.zip eva14:/share/chenshuaiwen-  
local/umi_data_raw
```

2. mp4+json 2 lerobot

服务器实现，需要uv的lerobot信息

chenshuaiwen-local下umi_data_raw

代码块

```
1 unzip data.zip
```

/home/chenshuaiwen/openpi/umi，在openpi环境下转换

代码块

```
1 python convert_mp4_data_to_lerobot_downsample_13.py --  
stage1_dir=/share/chenshuaiwen-local/umi_data_raw/batch_200 --  
repo=/share/chenshuaiwen-local/.cache/hf_home/fastumi --target_fps=10
```

训练

1. 对齐输入格式

src/openpi/policies下 xv13_policy

2. 改写config文件

需要一个data process config， 一个train config

train config写了数据路径和base model路径，还有一些训练参数

对应pi05_xv13_finetune

3. 计算norm

原版的norm是一个chunk内的所有action使用相同方式做norm

代码块

```
1 uv run scripts/compute_norm_stats.py --config-name pi05_xv13_finetune
```

为了满足relative action，需要考虑在chunk的每一个T分别做norm

4. 开训

注意开tmux，现在的tmux和exp_name相同

代码块

```
1  tmux new -s exp_name
2
3  tmux attach -t exp_name
```

注意改exp_name

JAX会默认使用可见的所有卡，所以需要修改可见卡数为当前可用的

代码块

```
1  CUDA_VISIBLE_DEVICES=0 XLA_PYTHON_CLIENT_MEM_FRACTION=0.9 uv run
   scripts/train.py pi05_xv13_finetune --exp-name=0107 --overwrite
```

注意一下loss情况

训完把norm_stats.json放在assets下

部署

需要把D435的数据也通过RealRL传过去，改realRL的数据传输和openpi的websocket_server，加入对传入数据的支持，需要rename到lerobot对应key

现在训好的模型是ckpt_xv_0127_w